

CSEN604: Data Bases II

Mohamed Karam Gabr

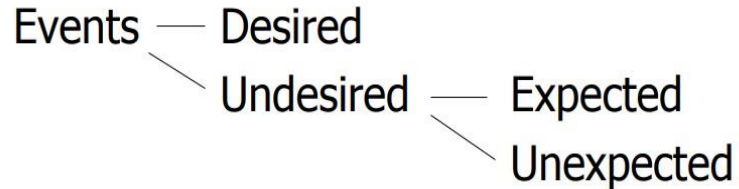
Logs and Recovery

Spring 2025

Outline

- Logging and recovery features in DB engines.
- Logging techniques:
 - Undo.
 - Redo.
 - Undo/Redo.

Failure Model



- Desired events:
 - What is intended and coded in SQL.
- Undesired expected events:
 - System crash due.
 - memory damaged.
 - CPU halts, resets.
 - Electrical current goes off.
- Undesired unexpected events:
 - Everything else.
 - Examples:
 - Disk data is lost or stolen.
 - Server power supply implodes wiping out universe.

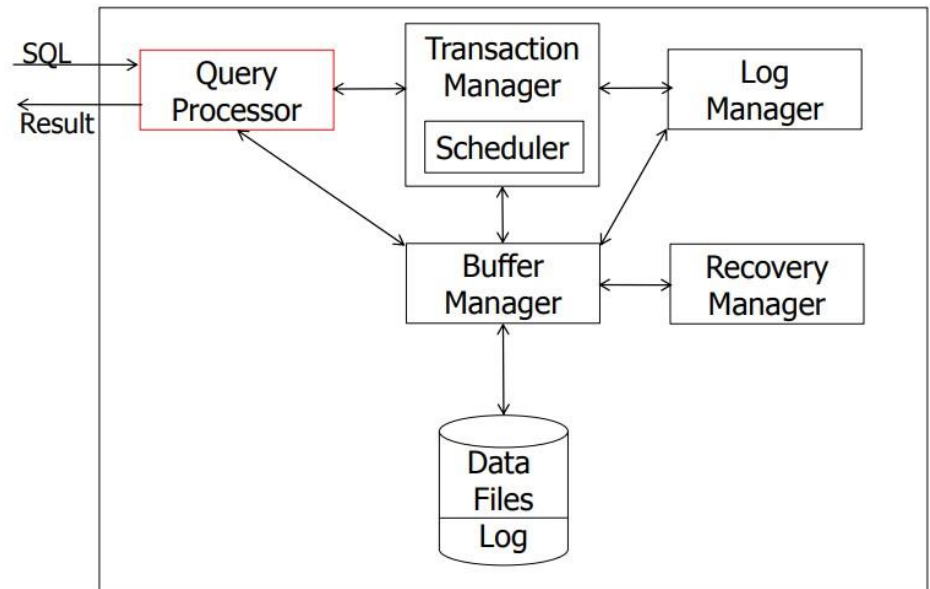
Operations

Operations between buffer manager and disk:

- Input (x): block containing x $\rightarrow$ memory
- Output (x): block containing x $\rightarrow$ disk

Operations between buffer manager and disk and then between transaction and buffer manager:

- Input (x): block containing x $\rightarrow$ memory
- Output (x): block containing x $\rightarrow$ disk
- Read (x,t): do input(x) if necessary
t $\leftarrow$ value of x in block
- Write (x,t): do output(x) if necessary
value of x in block $\leftarrow$ t



Example

Example

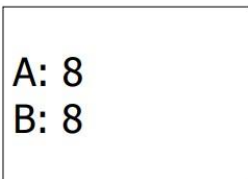
Constraint: $A=B$

$T_1: A \leftarrow A \times 2$

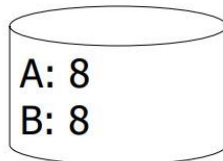
$B \leftarrow B \times 2$

```
start transaction;  
  update TableR set A = A* 2, B = B * 2;  
end transaction;
```

$T_1:$ Read (A,t); $t \leftarrow t \times 2$
Write (A,t);
Read (B,t); $t \leftarrow t \times 2$
Write (B,t);
Output (A);
Output (B);



memory



disk

Example

Example

Constraint: $A=B$

$T_1: A \leftarrow A \times 2$

$B \leftarrow B \times 2$

```
start transaction;  
  update TableR set A = A* 2, B = B * 2;  
end transaction;
```

T_1 : Read (A,t); $t \leftarrow t \times 2$
Write (A,t);
Read (B,t); $t \leftarrow t \times 2$
Write (B,t);
Output (A);
Output (B);

A: ~~8~~ 16
B: ~~8~~ 16

memory

A: 8
B: 8

disk

Example

Example

Constraint: $A=B$

$T_1: A \leftarrow A \times 2$

$B \leftarrow B \times 2$

```
start transaction;  
  update TableR set A = A* 2, B = B * 2;  
end transaction;
```

$T_1:$ Read (A,t); $t \leftarrow t \times 2$

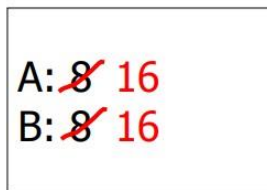
Write (A,t);

Read (B,t); $t \leftarrow t \times 2$

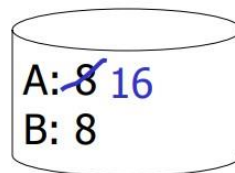
Write (B,t);

Output (A);

Output (B); failure!



memory



disk

Logging and Recovery Features in DB engines

Which transaction started?

What did it do? (or what it is going to do?)

Which transaction finished?

Log Commands:

<Start T>

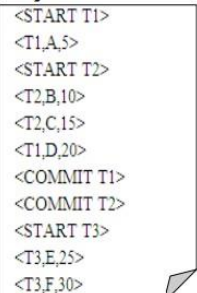
log the start of transaction T

<T, X, value>

log that T (transaction identifier) modified X
(database record) affecting value (value)

<COMMIT T>

log the completion of transaction T



```
<START T1>  
<T1,A,5>  
<START T2>  
<T2,B,10>  
<T2,C,15>  
<T1,D,20>  
<COMMIT T1>  
<COMMIT T2>  
<START T3>  
<T3,E,25>  
<T3,F,30>
```

Logging Techniques

- Undo logging: makes repairs to the database state by undoing the effects of transactions that may not have completed before the crash (brings database to an old consistent state).
- Redo logging: ignores incomplete transactions and repeats the changes made by committed transactions.
- Undo/Redo: a hybrid approach.

Undo Logging

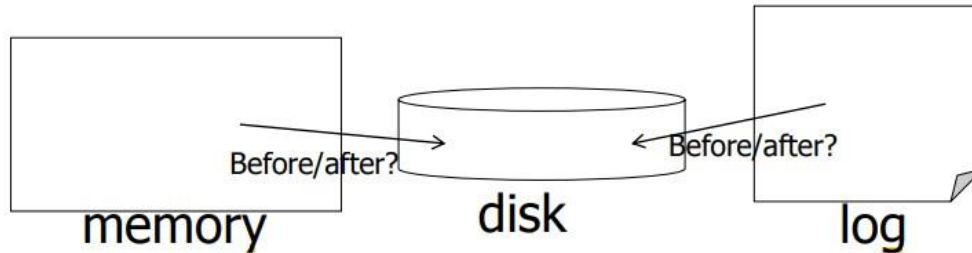
Undo Log Steps:

U1:

If transaction T modifies X, then the log record $\langle T, X, v \rangle$ must be written to disk **before** the new value of X is written to disk

U2:

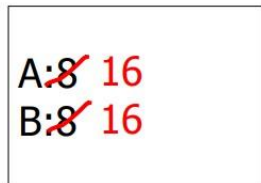
If a transaction commits, then its COMMIT log record must be written to disk only **after** database record written to disk.



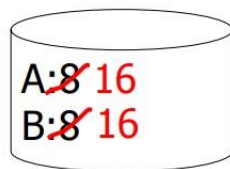
Undo Logging

Undo logging (Immediate modification)

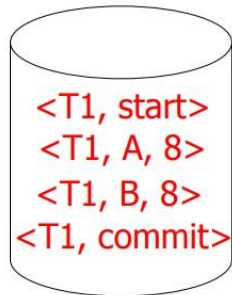
T1: Read (A,t); $t \leftarrow t \times 2$ A=B
 Write (A,t);
 Read (B,t); $t \leftarrow t \times 2$
 Write (B,t);
 Output (A);
 Output (B);



memory



disk



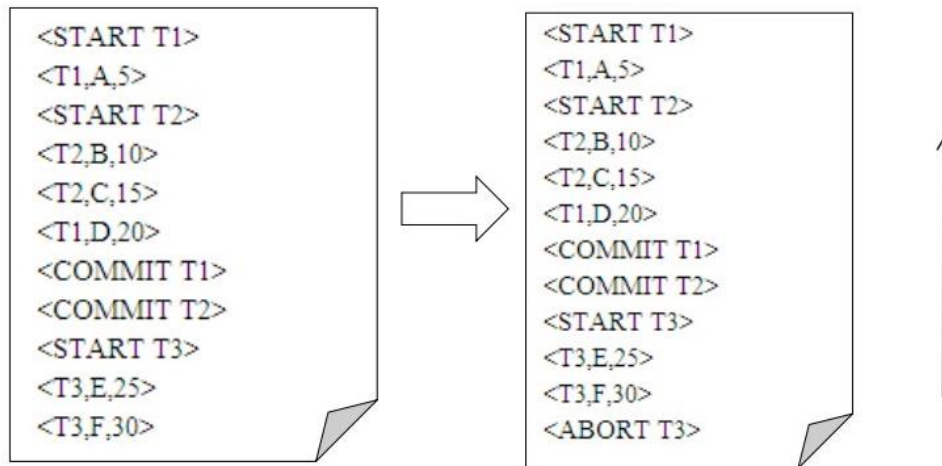
log

Step	Action	t	M-A	M-B	D-A	D-B	Log
1)							<START T>
2)	READ(A,t)	8	8		8	8	
3)	$t := t * 2$	16	8		8	8	
4)	WRITE(A,t)	16	16		8	8	<T,A,8>
5)	READ(B,t)	8	16	8	8	8	
6)	$t := t * 2$	16	16	8	8	8	
7)	WRITE(B,t)	16	16	16	8	8	<T,B,8>
8)	FLUSH LOG						
9)	OUTPUT (A)	16	16	16	16	8	
10)	OUTPUT (B)	16	16	16	16	16	
11)							<COMMIT T>
12)	FLUSH LOG						

Undo Logging

Recovery rules:

- 1) If T is a transaction whose COMMIT record has been seen, then do nothing. (T is committed and must not be undone)
- 2) Otherwise, T is an incomplete transaction, or an aborted transaction. The recovery manager change the value of X in the database to v (old value)



Undo Logging with Checkpoints

Stop accepting new transactions.

Wait until all currently active transactions commit or abort and have written a COMMIT or ABORT record on the log.

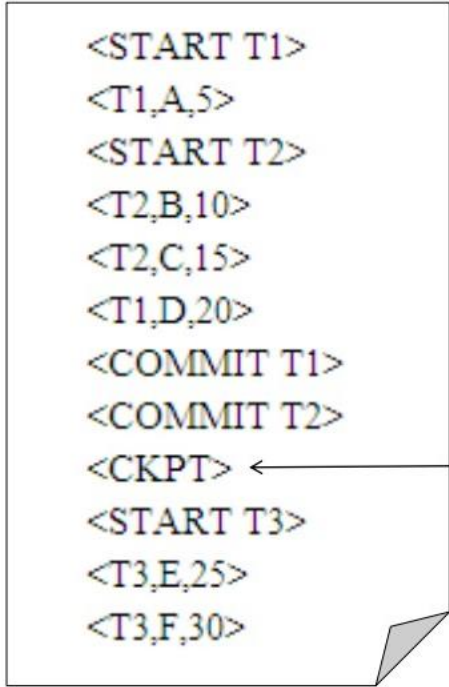
Flush the log to disk.

Write a log record <CKPT>, and flush the log again.

Resume accepting transactions.

How it works?

- 1) Queue new transactions
- 2) Wait until all running transactions commit
- 3) Flush the log
- 4) Write a log <CKPT>
- 5) Resume accepting transactions



```
<START T1>  
<T1,A,5>  
<START T2>  
<T2,B,10>  
<T2,C,15>  
<T1,D,20>  
<COMMIT T1>  
<COMMIT T2>  
<CKPT>  
<START T3>  
<T3,E,25>  
<T3,F,30>
```

Redo Logging

Redo Log Rule:

R₁:

Before modifying any database element X on disk, it is necessary that all log records pertaining to this modification of X, including **both** the **update** record $\langle T, X, v \rangle$ and the **<COMMIT T>** record, must appear on disk

Step	Action	t	M-A	M-B	D-A	D-B	Log
1)							<START T>
2)	READ(A,t)	8	8		8	8	
3)	$t := t * 2$	16	8		8	8	
4)	WRITE(A,t)	16	16		8	8	<T,A,16>
5)	READ(B,t)	8	16	8	8	8	
6)	$t := t * 2$	16	16	8	8	8	
7)	WRITE(B,t)	16	16	16	8	8	<T,B,16>
8)							<COMMIT T>
9)	FLUSH LOG						
10)	OUTPUT (A)	16	16	16	16	8	
11)	OUTPUT (B)	16	16	16	16	16	

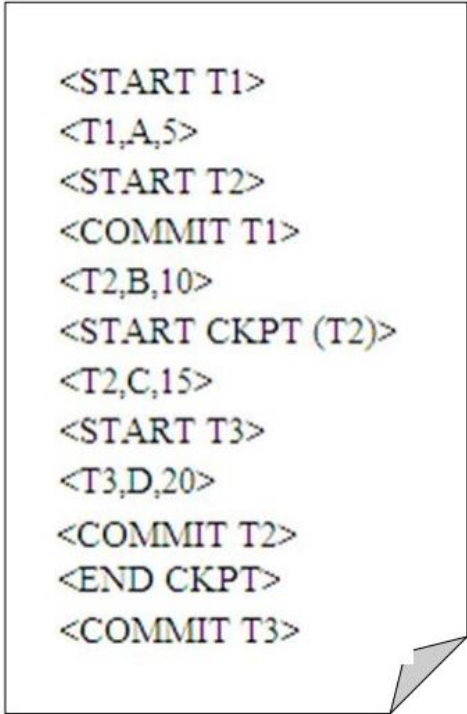
Redo Logging

Recovery rules:

- (1) Scan log from beginning. For each log record $\langle T, X, v \rangle$ encountered:
 - a. If T is not committed, do nothing
 - b. If T is committed, write value of v for database element X
- (2) For each incomplete transaction T , write an $\langle \text{ABORT } T \rangle$ record to the log

Redo Logging with Checkpoints

- (1) Write a log record <CKPT (T1,...Tk) for all active transactions
- (2) Flush the log
- (3) Write to disk all database elements that were written to memory buffers by transactions that had already committed when <CKPT> was inserted to log
- (4) Write <END CKPT>



```
<START T1>  
<T1,A,5>  
<START T2>  
<COMMIT T1>  
<T2,B,10>  
<START CKPT (T2)>  
<T2,C,15>  
<START T3>  
<T3,D,20>  
<COMMIT T2>  
<END CKPT>  
<COMMIT T3>
```

Undo Logging v Redo Logging

- *Undo logging*: cannot bring backup DB copies up to date
- *Redo logging*: need to keep all modified blocks in memory until commit

Undo/Redo Logging

- After a crash, the system consults the log to determine which transactions after the last checkpoint need to be redone and which need to be undone.
- Transaction T_i needs to be undone if the log contains the record $\langle T_i \text{ start} \rangle$ but does not contain either the record $\langle T_i \text{ commit} \rangle$ or the record $\langle T_i \text{ abort} \rangle$.
- Transaction T_i needs to be redone if log contains record $\langle T_i \text{ start} \rangle$ and either the record $\langle T_i \text{ commit} \rangle$ or the record $\langle T_i \text{ abort} \rangle$.

Undo/Redo Logging

Undo/Redo Log Rules:

UR₁:

Before modifying any database element X on disk because of changes made by some transaction T, it is necessary that the update record $\langle T, X, \text{old-val}, \text{new-val} \rangle$ appear on disk

Step	Action	t	M-A	M-B	D-A	D-B	Log
1)							<START T>
2)	READ(A,t)	8	8		8	8	
3)	t := t * 2	16	8		8	8	
4)	WRITE(A,t)	16	16		8	8	<T,A,8,16>
5)	READ(B,t)	8	16	8	8	8	
6)	t := t * 2	16	16	8	8	8	
7)	WRITE(B,t)	16	16	16	8	8	<T,B,8,16>
8)	FLUSH LOG						
9)	OUTPUT (A)	16	16	16	16	8	
10)							<COMMIT T>
11)	OUTPUT (B) FLUSH LOG	16	16	16	16	16	

Undo/Redo Logging

Recovery process:

- Backwards pass (end of log to latest valid checkpoint start)
 - construct set S of committed transactions
- Backward Pass
 - follow undo chains for transactions in (checkpoint active list) - S
- Forward pass (latest checkpoint start -> end of log)
 - redo actions of S transactions

